

Lecture 20

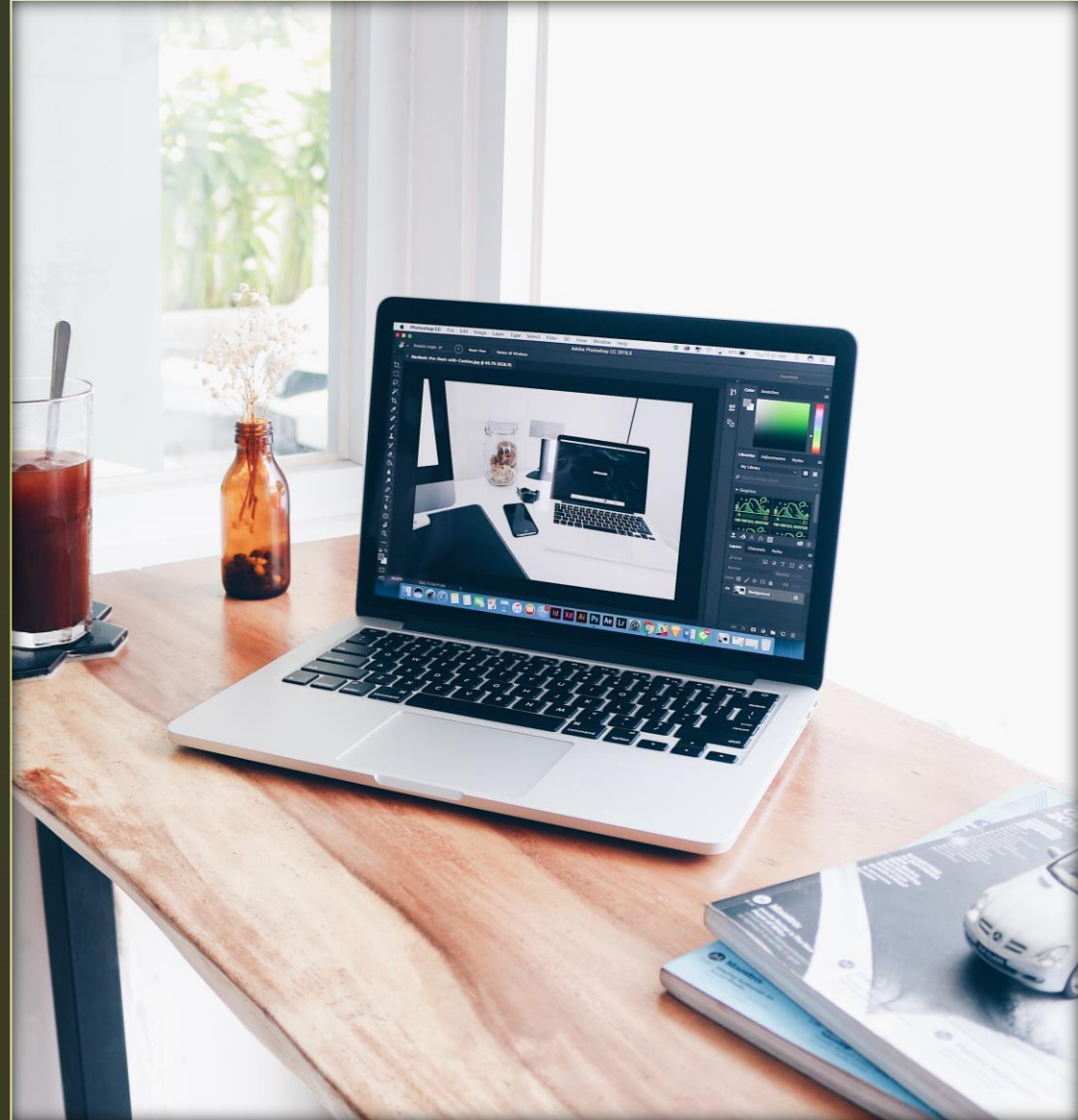
CS-317

Operating Systems



Book 1 Chapter 11

I/O Management



In the last lecture...

- We looked in detail at design issues involved in the I/O function of an OS.
- We discussed the logical structure of the I/O function.
- We understood the concept of buffering.
- We delved into different forms of buffering.
- We discussed disk scheduling.

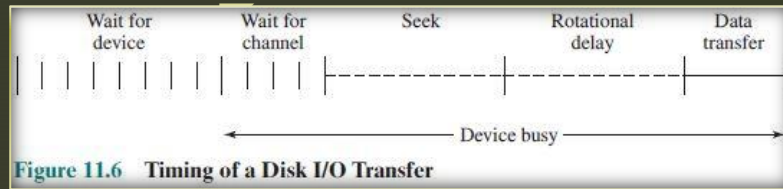




Disk Scheduling

Concept and algorithms





Disk performance parameters

- We have looked at different components of disk performance, which are:
 - Seek time,
 - Rotational delay,
 - Transfer time,
 - Additional I/O device and channel delays, and
 - RPS miss, if applicable.



A timing comparison

- Consider a disk with
 - Advertised average seek time of 4 milliseconds,
 - Rotation speed of 7,500 rpm, and
 - 512-byte sectors with 500 sectors per track.
- Suppose that we wish to read a file consisting of 2,500 sectors.



Case 1: The file is sequentially allocated.

- Let us assume that the file is stored as compactly as possible on the disk. This is known as sequential organization.

$$2,500 \text{ sectors} / 500 \text{ sectors per track} = 5 \text{ tracks}$$

- The time to read the first track is:

Average seek	4 milliseconds
--------------	----------------

Rotational delay	$(1 / (2 \times 7500)) \times 60 = 4$ milliseconds
------------------	--

Read 500 sectors	$2 \times 4 = 8$ milliseconds
------------------	-------------------------------

Total	16 milliseconds
-------	-----------------

- The remaining tracks can be read with no seek time.
- Each successive track is read in $4 + 8 = 12$ milliseconds.
- To read the entire file,

$$\text{Total time} = 16 + (4 \times 12) = 64 \text{ milliseconds}$$



Case 2: The file is randomly allocated.

- Now let us calculate the time required to read the same file saved using random rather than sequential allocation; that is, accesses to the sectors are distributed randomly over the disk.

- For each sector, we have:

Average seek	4 milliseconds
--------------	----------------

Rotational delay	4 milliseconds
------------------	----------------

Read 1 sector	$4 \text{ milliseconds} / (500 / 2) = 0.016 \text{ milliseconds}$
---------------	---

Total	8.016 milliseconds
-------	--------------------

- To read the entire file,

Total time = $2,500 \times 8.016 = 20,040$ milliseconds

- The order in which sectors are read from the disk has a tremendous effect on I/O performance.



Analysis of random scheduling

- Consider the typical situation in a multiprogramming environment, in which the OS maintains a queue of requests for each I/O device.
- For a single disk, there will be several I/O requests (reads and writes) in the queue, made by various processes.
- If we selected items from the queue in random order, then we can expect that the tracks will be visited randomly.
- The performance of the disk I/O system will be as poor as possible.
- This random scheduling is useful as a benchmark against which to evaluate other techniques.
- To improve matters, we need to reduce the average time spent on seeks.



> In a multiprogramming system, servicing disk requests in random order causes excessive seeks and very poor performance, so disk scheduling algorithms aim to reduce average seek time.

Disk scheduling policies

- We will discuss the following disk scheduling policies in detail:
 - First In First Out (FIFO)
 - Priority
 - Shortest Service Time First (SSTF)
 - SCAN
 - C-SCAN
 - N-step-SCAN
 - FSCAN



First In First Out (FIFO)

- The simplest form of scheduling is first-in-first-out (FIFO), which processes items from the queue in sequential order.
- This strategy is **fair**, because every request is honored in the order received.
- **If there are only a few processes that require access and if many of the requests are to clustered file sectors, then we can hope for good performance with FIFO.**
- This technique often approximates random scheduling in performance, if there are many processes competing for the disk.

This technique often approximates random scheduling in performance, if there are many processes competing for the disk.



Example request queue

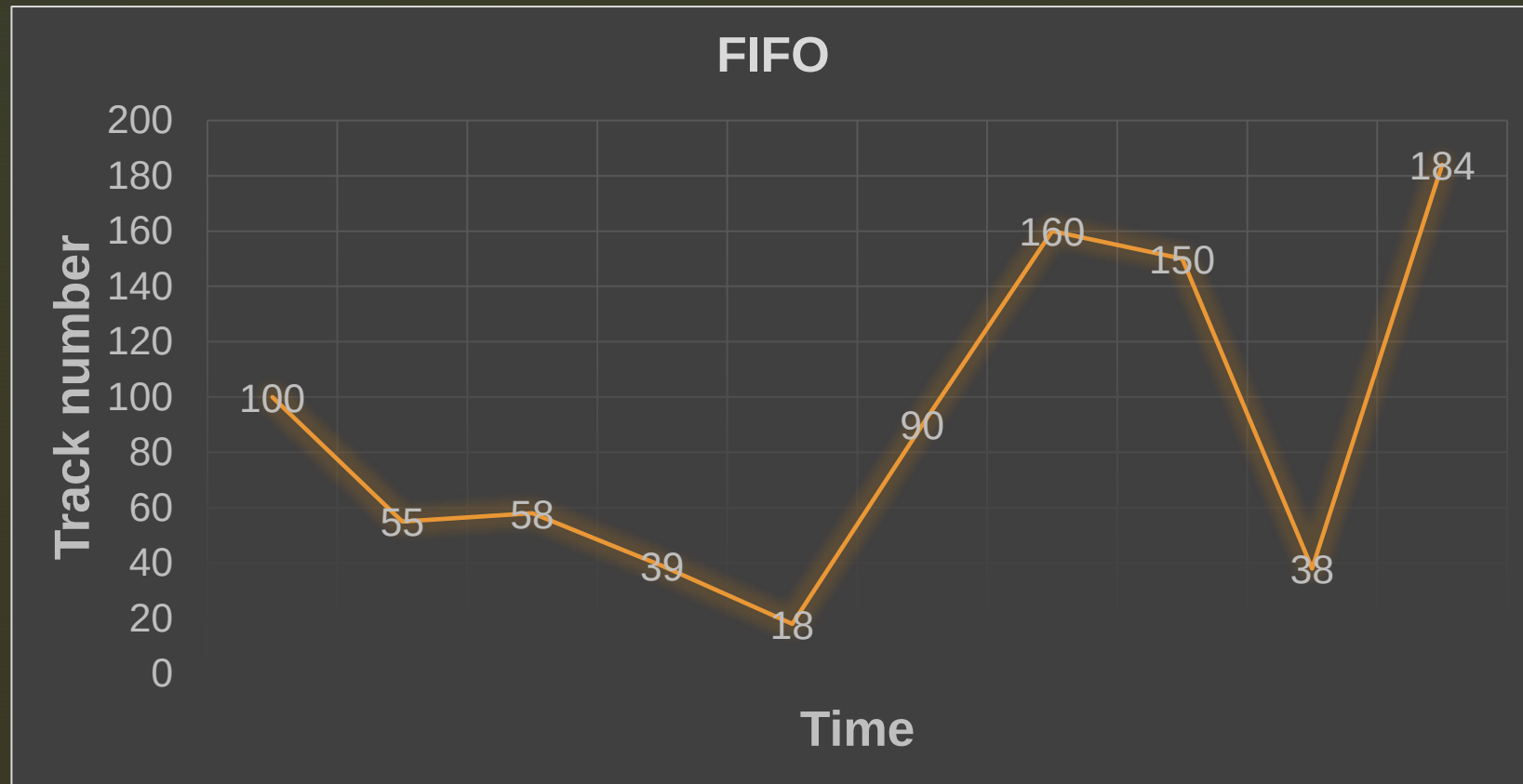
- A disk has 200 tracks (0 – 199).
- The disk head is initially located at track 100.
- The requested tracks, in the order received by the disk scheduler, are:
55, 58, 39, 18, 90, 160, 150, 38, 184.
- Use FIFO to schedule the above disk requests.

Starting track = 100

Next track	Tracks traversed
55	45
58	3
39	19
18	21
90	72
160	70
150	10
38	112
184	146
Total tracks	498
Average seek length	$498 / 9 = 55.33$



Graphical depiction of disk scheduling



Priority

- With a system based on priority, scheduling is outside the control of disk management software. The disk management software is fed requests according to the priority of the processes making them.
- This approach is not intended to optimize disk utilization but to meet other objectives within the OS.
- Often short batch jobs and interactive jobs are given higher priority than longer jobs that require longer computation. This allows a lot of short jobs to move through the system quickly and may provide good response time. However, longer jobs may have to wait excessively. To counter this policy, the users can split their jobs into smaller pieces and beat the system.
- This policy performs poorly for database systems, because successive requests are scattered across the disk, generating large average seek length.



Shortest Service Time First (SSTF)

- The SSTF policy is to select the disk I/O request that requires the least movement of the disk arm from its current position. Thus, we always choose to incur the minimum seek time.
- Of course, always choosing the minimum seek time does not guarantee that the average seek time over a number of arm movements will be minimum.
- However, this should provide better performance than FIFO.
- Because the arm can move in two directions, a random tie-breaking algorithm may be used to resolve cases of equal distances.



Example request queue

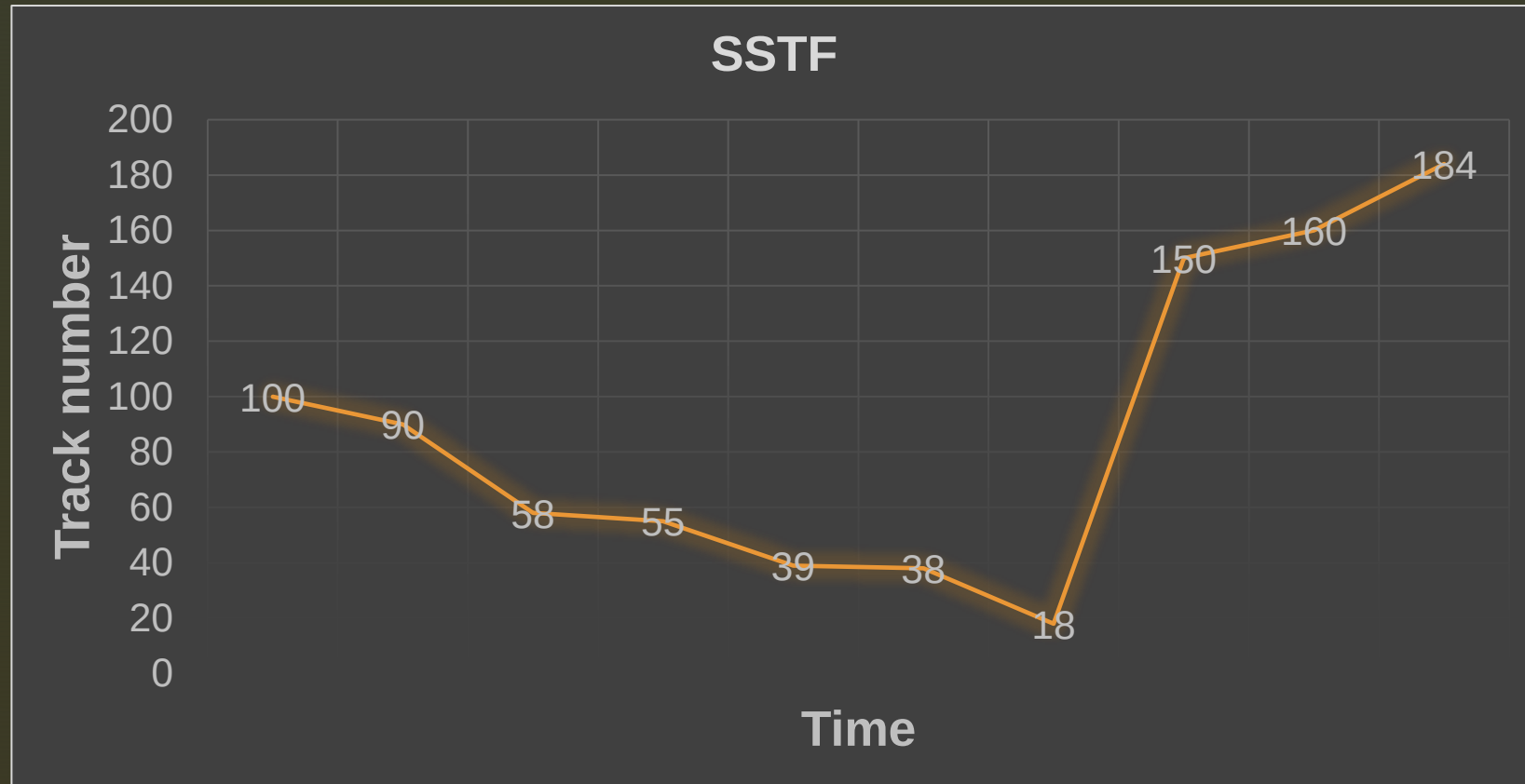
- A disk has 200 tracks (0 – 199).
- The disk head is initially located at track 100.
- The requested tracks, in the order received by the disk scheduler, are:
55, 58, 39, 18, 90, 160, 150, 38, 184.
- Use SSTF to schedule the above disk requests.

Starting track = 100	
Next track	Tracks traversed
90	10
58	32
55	3
39	16
38	1
18	20
150	132
160	10
184	24
Total tracks	248
Average seek length	$248 / 9 = 27.56$

find the point that will yield maximum traversal from the current, yet the last ones starve



Graphical depiction of disk scheduling



Problem with priority and SSTF

starvation can occur

- Both priority and SSTF can leave some request unfulfilled until the entire queue is emptied.
- There may always be the case when a newly arrived request is chosen to be served before an existing request in the queue.
- When the system constantly chooses incoming requests, the old requests in the queue start to starve.



- A simple alternative that prevents this sort of starvation is the SCAN algorithm, also known as the elevator algorithm because it operates much the way an elevator does.
- With SCAN, the arm is required to move in one direction only, satisfying all outstanding requests en route, until it reaches the last track in that direction.
- If the arm moves in one direction only, satisfying all outstanding requests en route, until there are no more requests in that direction, this refinement is sometimes referred to as the LOOK policy.
- The service direction is then reversed and the scan proceeds in the opposite direction, again picking up all requests in order.



Example request queue

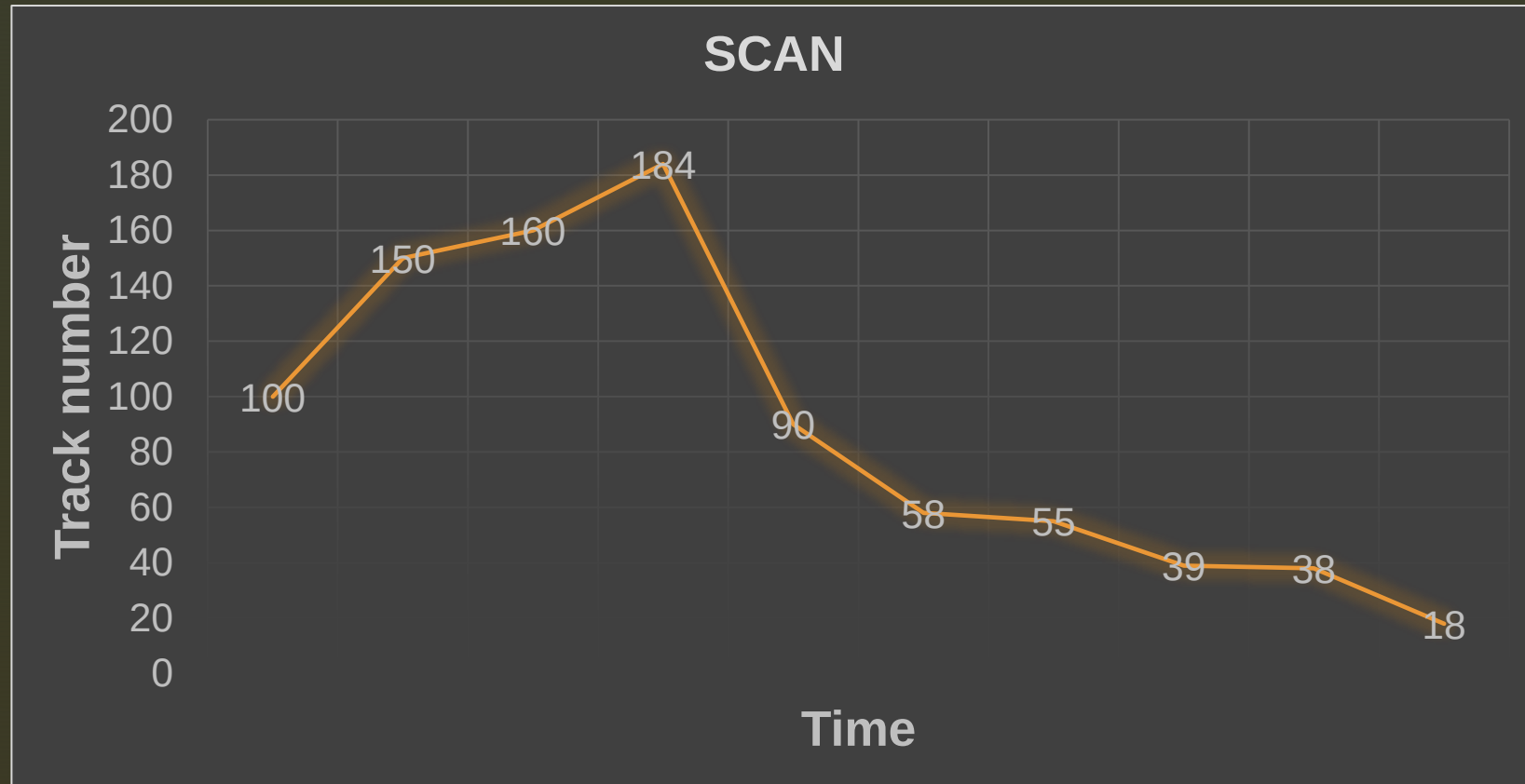
- A disk has 200 tracks (0 – 199).
- The disk head is initially located at track 100.
- The initial direction is of increasing track number.
- The requested tracks, in the order received by the disk scheduler, are:
55, 58, 39, 18, 90, 160, 150, 38, 184.
- Use SCAN (and LOOK) to schedule the above disk requests.

Starting track = 100

Next track	Tracks traversed
150	50
160	10
184	24
90	94
58	32
55	3
39	16
38	1
18	20
Total tracks	250
Average seek length	$250 / 9 = 27.78$



Graphical depiction of disk scheduling



Analysis of SCAN

- The **SCAN policy behaves almost identically to the SSTF policy.** Even when the queue is dynamically changing, SCAN will be like SSTF, unless the request pattern is unusual.
- **The SCAN policy is biased against the area most recently traversed. Thus it does not exploit locality as well as SSTF.**
- The SCAN policy favors jobs whose requests are for tracks nearest to both innermost and outermost tracks.
- Again in SCAN, old requests that are in the middle of the disk may starve, because newly arriving requests are located on tracks that SCAN favors.



C-SCAN

- The C-SCAN (circular SCAN) policy restricts scanning to one direction only. Thus, when the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again.
- This eliminates the bias of SCAN towards requests on tracks near the innermost or the outermost tracks of the disk.
- This reduces the maximum delay experienced by new requests on the other end of the disk.
- With SCAN, if the expected time for a scan from inner track to outer track is t , then the expected service interval for sectors at the periphery is $2t$. With C-SCAN, the interval is on the order of $t + s_{\max}$, where $s_{\max}$ is the maximum seek time.



Example request queue

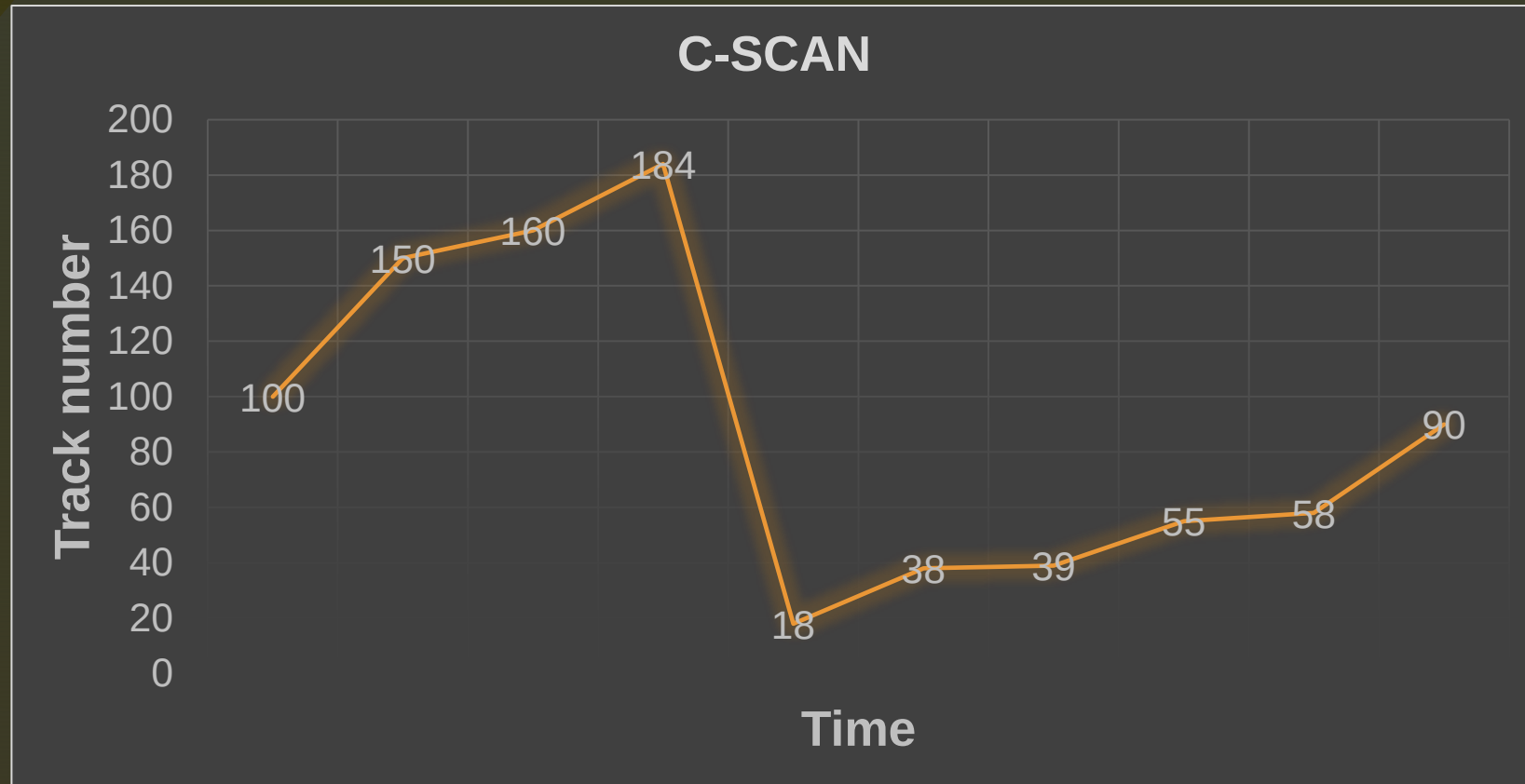
- A disk has 200 tracks (0 – 199).
- The disk head is initially located at track 100.
- The initial direction is of increasing track number.
- The requested tracks, in the order received by the disk scheduler, are:
55, 58, 39, 18, 90, 160, 150, 38, 184.
- Use C-SCAN to schedule the above disk requests.

Starting track = 100

Next track	Tracks traversed
150	50
160	10
184	24
18	166
38	20
39	1
55	16
58	3
90	32
Total tracks	322
Average seek length	$322 / 9 = 35.78$



Graphical depiction of disk scheduling

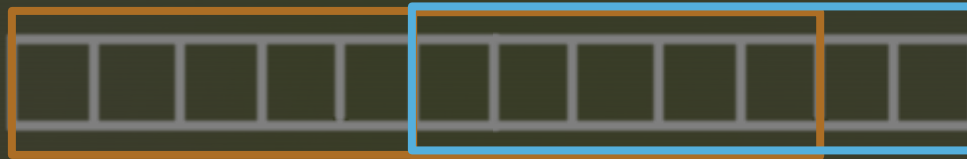


Problem with SSTF, CSAN and C-SCAN

- It is possible that the arm may not move for a considerable period.
- If one or a few processes have high access rates to one track, they can monopolize the entire device by repeated requests to that track.
- High-density multi-surface disks are more likely to be affected by this characteristic than lower-density disks and/or disks with only one or two surfaces.
- To avoid this "arm stickiness", the disk request queue can be segmented, with one segment at a time being processed completely.

If many requests keep arriving for the same nearby track(s), the disk arm may stay in one area for a long time (arm stickiness)





N-step-SCAN

- The N-step-SCAN policy segments the disk request queue into subqueues of length N. Subqueues are processed one at a time, using SCAN.
- While a queue is being processed, new requests must be added to some other queue.
- If fewer than N requests are available at the end of a scan, then all of them are processed with the next scan.
- With large values of N, the performance of N-step-SCAN approaches that of SCAN; with a value of $N = 1$, it degrades to FIFO.
- N-step-SCAN eliminates starvation of old requests at the hands of new requests, which we saw in SCAN.



FSCAN



- FSCAN is a policy that uses two subqueues.
- When a scan begins, all the requests are in one of the queues, with the other empty.
- During the scan, all new requests are put into the other queue.
- Thus, service of new requests is deferred until all the old requests have been processed.



Homework:

Use FIFO, SSTF, SCAN without LOOK, SCAN with LOOK, and C-SCAN to schedule these disk requests. Calculate the average seek length.

A disk has 200 tracks (0 – 199).

The disk head is initially located at track 100.

If required, the initial direction of the head should be assumed to be towards increasing track number.

The requested tracks, in the order received by the disk scheduler, are:

35, 47, 189, 170, 22, 134, 59, 12, 154, 129, 5, 67.





Disk Cache

Idea and implications



Cache memory

- The term cache memory is usually used to apply to a memory that is smaller and faster than main memory and that is interposed between main memory and the processor.
- Cache memory reduces average memory access time by exploiting the principle of locality.
- Locality of reference, also known as the principle of locality, is the tendency of a processor to access the same set of memory locations repetitively over a short period of time.



Disk cache

- The principle of locality can be applied to disk memory.
- A disk cache is a buffer in main memory for disk sectors. The cache contains a copy of some of the sectors on the disk.
- When an I/O request is made for a particular sector, a check is made to determine if the sector is in the disk cache. If so, the request is satisfied via the cache. If not, the requested sector is read into the disk cache from the disk.
- Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single I/O request, it is likely that there will be future references to that same block.



Design Considerations: Consequences of a disk cache hit

- When an I/O request is satisfied from the disk cache, the data in the disk cache must be delivered to the requesting process.
- This can be done either by:
 - Transferring the block of data within main memory from the disk cache to memory assigned to the user process, or
 - Simply by using a shared memory capability and giving a pointer to the appropriate slot in the disk cache to the asking process. This saves the time of a memory-to-memory transfer and allows shared access by processes using the readers / writers model. The data in the disk cache is a shared resource. The processes asking for it can either read it or write on it.



Design Considerations: Replacement strategy - LRU

- When a new sector is brought in, an existing block may need to be replaced.
- The most commonly used algorithm is Least Recently Used (LRU). LRU dictates to replace that block that has been in the cache longest with no reference to it.
- Logically, the cache consists of a stack of blocks, with the most recently referenced block on the top of the stack.
- When a block in the cache is referenced, it is moved from its existing position to the top of the stack.
- When a block is brought in from secondary memory, we remove the block that is at the bottom of the stack from the disk cache to make space for the new block and push the incoming block onto the top of the stack.



Design Considerations: Replacement strategy - LFU

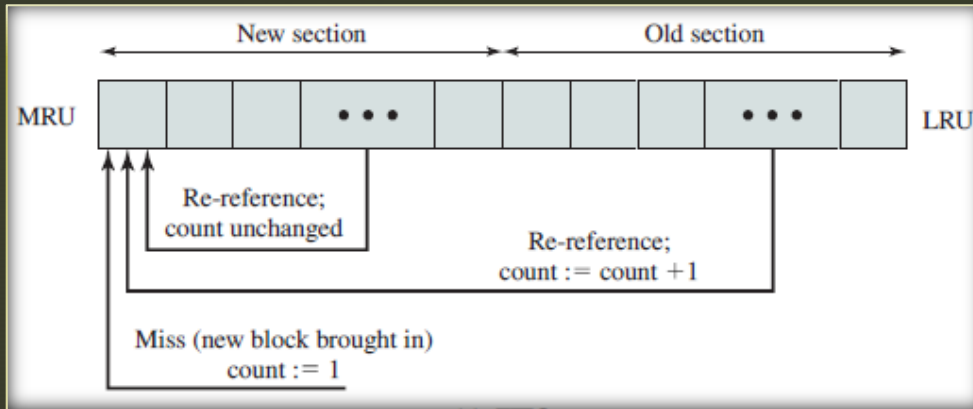
- Another possibility is Least Frequently Used (LFU): Replace that block in the set that has experienced the fewest references.
- LFU is implemented by associating a counter with each block.
- When a block is brought in, it is assigned a count of 1.
- With each reference to the block, its count is incremented by 1.
- When replacement is required, the block with the smallest count is selected.



LFU and LRU: Which is better?

- Intuitively, LFU may seem more appropriate than LRU, because LFU makes use of more pertinent information about each block in the selection process.
- A simple LFU algorithm has the following problem. It may be that certain blocks are referenced rarely overall, but when they are referenced, there are short intervals of repeated references due to locality, thus building up high reference counts. After such an interval is over, the reference count may be misleading. It will reflect that the block will be referenced again, but in truth, the block will not be used further.
- This causes LFU to make poor replacement choices.

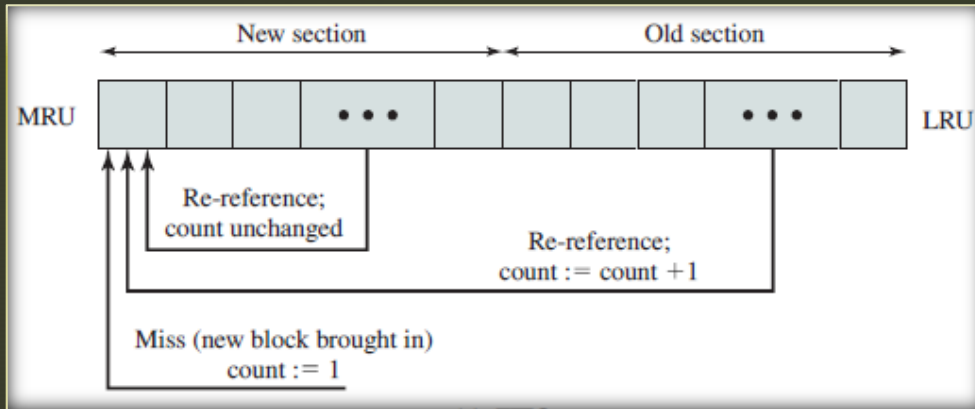




Frequency-based replacement (FBR)

- The blocks are logically organized in a stack.
- A certain portion of the top part of the stack is designated the new section.
- When there is a cache hit, the referenced block is moved to the top of the stack.
- If the block was already in the new section, its reference count is not incremented; otherwise it is incremented by 1.
- Blocks in the new section are never replaced.
- This strategy achieves only slight improvement over LRU.

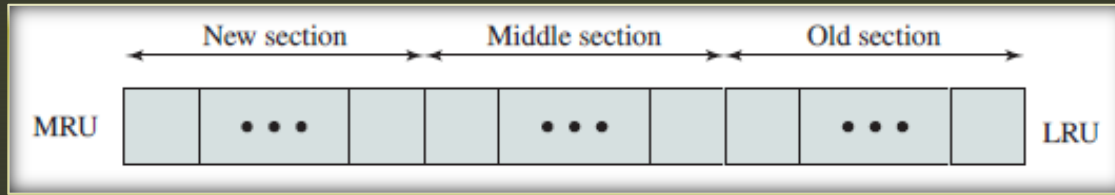




Problem with FBR

- On a cache miss, when a new block is brought into the new section, its frequency count is 1.
- The count remains at 1 as long as the block remains in the new section.
- Eventually the block ages out of the new section, i.e. it gets pushed back until it becomes part of the old section, with its count still at 1.
- If the block is not re-referenced fairly quickly, it is very likely to be replaced because it has the smallest reference count of those blocks that are not in the new section.
- In other words, before blocks pushed back into the old section can be re-referenced and can get a count greater than 1, they are replaced because they are part of the old section.





Refinement of FBR

- Divide the stack into three sections: new, middle, and old.
- Reference counts are not incremented on blocks in the new section.
- Only blocks in the old section are eligible for replacement.
- Assuming a sufficiently large middle section, this allows relatively frequently referenced blocks a chance to build up their reference counts before becoming eligible for replacement, i.e. before they are pushed back into the old section.
- Simulation studies indicate that this refined policy is significantly better than simple LRU or LFU.



Replacement and write back

- The replacement of a block can be:
 - On-demand: a sector is replaced only when the slot is needed.
 - Preplanned: several slots are released at a time.
- If a sector is brought into the cache and only read, it is not necessary to write it back when replacing it. If a sector is brought into the cache and written, it is necessary to write it back when replacing it.
- Preplanned replacement happens when we need to write back sectors. We cluster the writing and use a scheduling policy to minimize seek time.



Performance considerations

- The issue of cache performance reduces itself to a question of whether a given miss ratio can be achieved.
- This will depend on the locality behavior of the disk references, the replacement algorithm, and other design factors.
- Principally, the miss ratio is a function of the size of the disk cache. The larger the disk cache, the lower is its miss ratio.
- The exact sequence of reference patterns, plus related design issues such as block size, have a profound influence on the performance achieved by a replacement strategy.



Reading assignment

Read article 11.6: RAID

Redundant Array of Independent Disks



What did we gather so far?

- We conducted timing comparison of data transfers for different file allocations.
- We looked at various disk scheduling policies and analyzed them.
- We were introduced to the concept of disk cache.
- We looked at the design and performance considerations of disk cache.



Things to do

Read

Read the book.



Note

Write your questions on a piece of paper and keep it in front of you during the online session.



Email

If you have suggestions relevant to content or quality of this lecture, email them to me.

